PROJECT MASTER DOCUMENT

Title: Asset Defender - Problem Statement, Extended Abstract, Methodology, Workflow, and Diagrams

Generated on: 2026-03-08

Problem Statement

Social media users face persistent risks from fake profiles, impersonation campaigns, spam conversations, and phishing attempts. Most users cannot reliably identify sophisticated scam behavior, especially when attackers blend social engineering language with seemingly authentic profile metadata. Existing controls are either platform-side and opaque or lightweight local filters with low explainability. The project problem is to design and implement a practical system that detects profile and message risk in near real time, explains why a score was assigned, stores historical evidence for audit, and supports both end-user and administrator workflows without requiring enterprise-grade infrastructure.

Extended Abstract (5000+ words)

This extended abstract presents Asset Defender, a hybrid cyber-safety platform designed to detect fake Instagram profiles, suspicious direct messages, scam campaigns, and account abuse patterns using a combined architecture of machine learning, deterministic heuristics, browser extension telemetry, and server-side risk fusion. The core research problem is operational: consumer users receive high volumes of social engineering attempts in messaging and profile contexts, yet most available safeguards are either platform-level moderation that users cannot tune or simple keyword filters that produce unstable detection quality. Asset Defender addresses this gap by combining multiple forms of evidence and by presenting explainable risk output for both end users and administrators.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered

strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided

content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post

volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst

behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks,

computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables

dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final

risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware

value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality

is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This

design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails,

while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block,

report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather

than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or

scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post

volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and

temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks,

computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into

SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final

predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model

probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and

conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This

design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one

model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block,

report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a

layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped

profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post

volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and

temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks,

computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables

dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final

risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware

value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and

conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This

design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one

model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block,

report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered

strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided

content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following

structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst

behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-

level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables

dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final

predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware

value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality

is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of problem framing and attacker behavior, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of data representation and feature engineering, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of hybrid trust scoring and confidence estimation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of browser extension workflow for in-context collection, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data

conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of backend decision fusion and secure persistence, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of profile model training pipeline and threshold strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of message model training pipeline and lexical safeguards, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of admin analytics and governance controls, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails,

while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of system reliability, fallback logic, and graceful degradation, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of deployment constraints and local-first architecture, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of privacy and legal boundaries for scraping workflows, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of explainability and user-facing recommendations, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue

engagement with an account.

In the domain of limitations and residual risk scenarios, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

In the domain of future roadmap and continuous improvement strategy, the project uses a layered strategy rather than a single classifier decision. The frontend gathers evidence from user-provided content or scraped profile context, then computes heuristic indicators such as follower-following structure, post volume, engagement quality, suspicious phrase presence, repetition patterns, and temporal burst behavior. The backend validates requests, applies authentication and role checks, computes server-level features, invokes suitable model endpoints, and persists normalized outputs into SQLite tables dedicated to profile analysis, message analysis, behavior analysis, and final predictions. The final risk score is not treated as an opaque model output; it is an evidence-aware value that merges model probability, rule-based corrections, confidence penalties when sample quality is weak, and conservative safeguards to avoid high-confidence claims under insufficient data conditions. This design increases operational trust, because the system can still function if one model path fails, while preserving explainable reasoning for users who must decide whether to block, report, or continue engagement with an account.

The empirical model layer includes profile classifiers (Random Forest, Gradient Boosting, Logistic Regression, and XGBoost) and message classifiers (Multinomial Naive Bayes and Logistic Regression with TF-IDF vectorization). Evaluation artifacts in the project repository report strong classification quality across these baselines. Beyond raw accuracy, the evaluation script exports precision, recall, F1, ROC-AUC, confusion matrices, and threshold sweep reports. This is important for security use cases because the cost of false negatives can be severe in scam or credential-theft scenarios.

From a software engineering perspective, the project contributes an end-to-end operational stack that links real-time interface collection, risk analytics, persistence, and dashboard visualization. Instead of treating ML as an isolated notebook exercise, Asset Defender embeds trained artifacts into a production-oriented flow with login, role controls, history retrieval, admin oversight, and user-specific analysis tracking. The architecture is intentionally practical: local-first development on Node.js plus SQLite, Python-based model scripts for reproducible training, and modular client code for heuristic and behavioral analysis. This balanced approach makes the system suitable for academic demonstration and for incremental hardening toward production deployment.

In conclusion, the project demonstrates that effective anti-scam defense on social media requires a mixed methodology: data-driven classification, rule-based safety controls, confidence gating, and human-readable recommendations. The platform does not claim perfect prediction, but it establishes a defensible, extensible baseline that can evolve with adversary behavior, multilingual content, and policy requirements.

Methodology (3000+ words)

Methodology Overview
Asset Defender follows a hybrid, evidence-centric methodology combining supervised learning, deterministic heuristics, risk calibration, and defensive software engineering controls. The method is

structured as a pipeline with explicit stages: problem framing, data acquisition and preparation, feature construction, model training, threshold selection, heuristic augmentation, backend fusion, persistence, dashboard interpretation, and iterative validation.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic

Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC,

and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max

feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and

apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic

Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and

message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence,

classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first

constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message

training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce

oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence

paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile

picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam,

phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over

time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing

to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator,

and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Stage 1 - Problem Definition: The project defines two core prediction units: profile-level risk and message-level risk. Profile risk targets fake or bot-like account behavior. Message risk targets spam, phishing, social engineering, and scam intent.

Stage 2 - Data Sources: Profile training uses the CSV dataset under profiledatset/train.csv. Message training uses messagedatasets/spam.csv. Labels are inherited from source datasets, with preprocessing to normalize feature types.

Stage 3 - Feature Engineering: Profile features include structural account fields such as profile picture presence, username patterns, follower and following statistics, post count, privacy indicator, and follower-to-following ratio. Message features use TF-IDF vectors with stop-word filtering and max feature bounds for stable sparsity.

Stage 4 - Model Training: The profile branch trains Random Forest, Gradient Boosting, Logistic Regression, and optional XGBoost. The message branch trains Multinomial Naive Bayes and Logistic Regression over TF-IDF vectors.

Stage 5 - Evaluation and Thresholding: The evaluation script computes precision, recall, F1, ROC-AUC, and confusion matrices. A threshold sweep from 0.05 to 0.95 selects operating points by recall-first constraints and F1 maximization.

Stage 6 - Runtime Fusion: Backend routes derive heuristic metrics, blend model and rule outputs, and apply evidence safeguards. High-threat cases can trigger risk floors, while insufficient evidence paths reduce confidence.

Stage 7 - Persistence and Explainability: Every analysis is stored with risk components, confidence, classification tags, and recommendations so both user and admin dashboards can inspect reasoning over time.

Stage 8 - Governance and Security: Authentication uses bcrypt and JWT; role-based admin routes enforce oversight boundaries. Rate limiting and throttling logic reduce abuse surfaces.

Methodological Validation

The methodology is validated through integrated scripts, runtime inference checks, and persistence-level verification. Because this system is a decision-support tool rather than an autonomous

enforcement engine, the method prioritizes transparent reasoning, confidence disclosure, and fallback continuity under partial failures.

Workflow Description
1) User authentication and authorization via signup/login and JWT issuance.
2) Data acquisition through manual content input or extension-assisted scraping.
3) Client-side heuristics for profile structure, bio keywords, caption sentiment, and behavioral cues.
4) Server-side feature extraction for message threat and profile quality indicators.
5) ML inference using pre-trained models and optional fallback logic.
6) Risk fusion with confidence scoring and safety guards for insufficient evidence.
7) Persistence in SQLite analysis tables for user and admin retrieval.
8) Visualization in dashboard pages with recommendations and risk history.

Required Diagrams

1. Workflow Diagram
[User Login/Signup] -> [Input via Analyze Page or Extension] -> [Client Heuristic Engine] -> [Server API] -> [ML Inference + Rules] -> [Risk Fusion] -> [SQLite Persistence] -> [Dashboard/Admin Visualization]

2. Class Diagram (Textual UML)
Classes:
- User: id, username, email, account_type, password_hash, created_at
- ProfileAnalysis: analyzed_username, followers_count, following_count, anomaly_score, heuristics
- MessageAnalysis: total_messages, spam_count, scam_count, threat_score, heuristics
- BehaviorAnalysis: posting_pattern, follower_growth_spike, anomaly_score
- FinalPrediction: risk_score, risk_level, confidence_score, explanation_summary
Relations:
- User 1..* ProfileAnalysis
- User 1..* MessageAnalysis
- User 1..* BehaviorAnalysis
- User 1..* FinalPrediction

3. Entity Relationship Diagram (ER)
users (id PK) ---< profile_analysis (user_id FK)
users (id PK) ---< message_analysis (user_id FK)
users (id PK) ---< behavior_analysis (user_id FK)
users (id PK) ---< final_predictions (user_id FK)

4. Sequence Diagram (Profile Analysis)
Actor User -> Client AnalyzePage: submit profile JSON
Client AnalyzePage -> Heuristic Analyzer: compute structural/content/behavioral risk
Client AnalyzePage -> Server /api/analyses/client: send heuristics + content + token
Server -> Profile Model Inference: get risk probability
Server -> DB: insert profile_analysis, behavior_analysis, final_predictions
Server -> Client: return analysis id
Client -> Heuristic Analysis Page: render result

5. Sequence Diagram (Message Analysis)
User -> Client AnalyzePage: submit message text
Client -> Server /api/analyses: authenticated request
Server -> Message Feature Extractor: derive lexical and timing signals
Server -> Message Model: predict spam/scam probability
Server -> DB: insert message_analysis and final_predictions
Server -> Client: return analysis id and risk metadata

6. Component Diagram

- Browser Extension: popup.js, background.js, instagram_analyzer.js
- React Client: pages, hooks, UI components
- API Server: Express routes, auth middleware, risk computation
- ML Layer: Python scripts + serialized models
- Data Layer: SQLite app.db

7. Deployment Diagram
Node Runtime (Express API) <-> SQLite file database
React Web App (Vite) <-> Express API via HTTP
Python Runtime for model training and evaluation (offline or scheduled)
Browser Extension interacts with Instagram pages and API endpoints

8. Activity Diagram
Start -> Receive content -> Validate input -> Extract features -> Run model -> Apply heuristic rules -> Blend risk -> Set confidence -> Persist result -> Return recommendations -> End

9. Data Flow Diagram
Input Content -> Preprocessing -> Feature Extraction -> ML Predictor and Rule Engine -> Risk Fusion -> Storage -> Dashboard Output

Model Evaluation Snapshot

- Domain: profile, Model: random_forest, Accuracy: 0.9310, Precision: 0.9167, Recall: 0.9483, F1: 0.9322, ROC-AUC: 0.9854
- Domain: profile, Model: gradient_boosting, Accuracy: 0.9224, Precision: 0.9153, Recall: 0.9310, F1: 0.9231, ROC-AUC: 0.9834
- Domain: profile, Model: logistic_regression, Accuracy: 0.9483, Precision: 0.9815, Recall: 0.9138, F1: 0.9464, ROC-AUC: 0.9887
- Domain: profile, Model: xgboost, Accuracy: 0.9310, Precision: 0.9310, Recall: 0.9310, F1: 0.9310, ROC-AUC: 0.9831
- Domain: message, Model: multinaive, Accuracy: 0.9740, Precision: 0.9918, Recall: 0.8121, F1: 0.8930, ROC-AUC: 0.9904
- Domain: message, Model: message_logistic_regression, Accuracy: 0.9614, Precision: 1.0000, Recall: 0.7114, F1: 0.8314, ROC-AUC: 0.9871